

Playwright Cheat Sheet

The complete quick-reference for Playwright test automation

[Locators](#) · [Actions](#) · [Assertions](#) · [Waiting](#) · [Network](#) · [Debugging](#) · [CLI](#) · [Best Practices](#)

What's inside

- 1. Installation & Project Setup
- 2. Writing Tests — Basics
- 3. Locators — Finding Elements
- 4. Actions
- 5. Assertions (Web-First, Auto-Retrying)
- 6. Waiting Strategies
- 7. Navigation, Tabs, Frames & Windows
- 8. Network Interception & Mocking
- 9. Debugging & Tooling
- 10. CLI & Config Essentials
- 11. Best Practices

Installation & Project Setup

```
npm init playwright@latest
```

Scaffold a new Playwright project (config, examples, CI workflow).

```
npx playwright install
```

Download browser binaries (Chromium, Firefox, WebKit).

```
npx playwright install --with-deps
```

Install browsers plus required OS-level dependencies (Linux/CI).

```
npx playwright --version
```

Print the installed Playwright version.

Writing Tests — Basics

```
import { test, expect } from  
'@playwright/test'
```

Standard imports for every spec file.

```
test('name', async ({ page }) => { ...  
})
```

Define a test with a fresh page/browser context.

```
test.describe('group', () => { ... })
```

Group related tests together.

```
test.beforeEach / test.afterEach
```

Run setup/teardown logic around each test.

```
test.only / test.skip / test.fixme
```

Focus, skip, or mark a test as broken.

```
test.step('desc', async () => { ... })
```

Label a logical step for clearer test reports.

Locators — Finding Elements

<code>page.getByRole('button', { name: 'Submit' })</code>	Preferred locator — matches accessible role & name.
<code>page.getByText('Welcome')</code>	Match by visible text content.
<code>page.getByLabel('Email')</code>	Match a form control by its associated <label>.
<code>page.getByPlaceholder('Search...')</code>	Match an input by its placeholder text.
<code>page.getByTestId('submit-btn')</code>	Match by data-testid attribute (configurable).
<code>page.getByTitle('Close')</code>	Match by the title attribute.
<code>page.getByAltText('Logo')</code>	Match an by its alt text.
<code>page.locator('css=selector')</code>	Fallback CSS selector locator.
<code>locator.filter({ hasText: 'X' })</code>	Narrow a locator by inner text.
<code>locator.nth(0) / .first() / .last()</code>	Pick a specific match from multiple elements.

Actions

<code>await locator.click()</code>	Click an element (auto-waits for actionability).
<code>await locator.dblclick()</code>	Double-click an element.
<code>await locator.fill('text')</code>	Clear and type text into an input/textarea.
<code>await locator.type('text')</code>	Type text character by character (rarely needed vs fill).
<code>await locator.press('Enter')</code>	Send a single key press.
<code>await locator.check() / .unchecked()</code>	Set a checkbox/radio to checked or unchecked.
<code>await locator.selectOption('value')</code>	Choose an option in a <select>.
<code>await locator.hover()</code>	Move the mouse over an element.
<code>await locator.dragTo(target)</code>	Drag-and-drop onto another locator.
<code>await locator.setInputFiles('path')</code>	Upload one or more files via a file input.
<code>await locator.focus() / .blur()</code>	Move or remove keyboard focus.
<code>await locator.scrollIntoViewIfNeeded()</code>	Scroll the element into the viewport.

Assertions (Web-First, Auto-Retrying)

<code>await expect(locator).toBeVisible()</code>	Element is visible in the DOM/viewport.
<code>await expect(locator).toBeHidden()</code>	Element is hidden or detached.
<code>await expect(locator).toBeEnabled() / toBeDisabled()</code>	Element's enabled/disabled state.
<code>await expect(locator).toBeChecked()</code>	Checkbox/radio is checked.
<code>await expect(locator).toHaveText('x')</code>	Element's text content equals/matches.
<code>await expect(locator).toContainText('x')</code>	Element's text content contains a substring.
<code>await expect(locator).toHaveValue('x')</code>	Input/textarea/select has this value.
<code>await expect(locator).toHaveCount(3)</code>	Locator resolves to exactly N elements.
<code>await expect(locator).toHaveAttribute('a', 'v')</code>	Element has attribute with given value.
<code>await expect(locator).toHaveCSS('prop', 'v')</code>	Element has a computed CSS property value.
<code>await expect(page).toHaveURL(/regex/)</code>	Current page URL matches.
<code>await expect(page).toHaveTitle('x')</code>	Document title matches.
<code>await expect(locator).toHaveScreenshot()</code>	Visual regression snapshot comparison.

Waiting Strategies

Auto-waiting

Actions & assertions auto-wait/retry — avoid manual sleeps.

```
await locator.waitFor({ state:  
'visible' })
```

Wait for a specific element state.

```
await page.waitForURL('**/success')
```

Wait until navigation reaches a URL pattern.

```
await  
page.waitForLoadState('networkidle')
```

Wait for the page to settle (use sparingly).

```
await page.waitForResponse(url)
```

Wait for a matching network response.

```
await page.waitForRequest(url)
```

Wait for a matching outgoing network request.

```
await page.waitForTimeout(ms)
```

Hard-coded delay — last resort, avoid in real tests.

Navigation, Tabs, Frames & Windows

<code>await page.goto('https://example.com')</code>	Navigate the page to a URL.
<code>await page.goBack() / page.goForward()</code>	Browser history navigation.
<code>const [newPage] = await Promise.all([context.waitForEvent('page'), locator.click()])</code>	Capture a new tab/window opened by an action.
<code>const frame = page.frameLocator('iframe').locator('x')</code>	Query inside an iframe via frameLocator.
<code>await context.newPage()</code>	Open an additional tab in the same browser context.
<code>await page.bringToFront()</code>	Focus a specific tab.

Network Interception & Mocking

```
await page.route('**/api/**', route =>
  route.fulfill({...}))
```

Mock or modify a network response.

```
await page.route('**/*.png', route =>
  route.abort())
```

Block matching requests (e.g. images).

```
await page.unroute('**/api/**')
```

Remove a previously registered route handler.

```
const res = await page.request.get(url)
```

Make a standalone API request (no browser needed).

```
await context.route(...)
```

Apply routing rules across an entire browser context.

Debugging & Tooling

```
npx playwright test --headed
```

Run tests with a visible browser window.

```
npx playwright test --debug
```

Step through a test with the Playwright Inspector.

```
npx playwright codegen <url>
```

Record actions and generate test code.

```
await page.pause()
```

Pause execution and open the Inspector mid-test.

```
npx playwright show-trace trace.zip
```

Explore a recorded trace timeline visually.

```
PWDEBUG=1
```

Environment variable to force debug mode.

```
await page.screenshot({ path: 'x.png' })
```

Capture a screenshot for debugging.

CLI & Config Essentials

<code>npx playwright test</code>	Run the full test suite.
<code>npx playwright test file.spec.ts</code>	Run a single spec file.
<code>npx playwright test -g "pattern"</code>	Run only tests matching a name pattern.
<code>npx playwright test --project=chromium</code>	Run tests for a specific configured project.
<code>npx playwright show-report</code>	Open the last HTML test report.
<code>npx playwright test --workers=4</code>	Control the number of parallel workers.
<code>use: { baseURL, headless, trace, video }</code>	Common playwright.config.ts test options.
<code>projects: [{ name, use: { ...devices['...'] } }]</code>	Configure multiple browsers/devices.

Best Practices

Prefer user-facing locators

getByRole/getByText/getByLabel over brittle CSS/XPath.

Avoid `waitForTimeout`

Rely on auto-waiting assertions and locator actions instead.

Keep tests independent

Each test should set up its own state; avoid shared mutable state.

Use fixtures

Share setup logic (auth, data) via custom test fixtures.

Enable trace on retry

trace: 'on-first-retry' makes failures easy to debug in CI.

Isolate with browser contexts

One context per test/session for clean isolation.